KAN it learn? Interpretable N-Body Physics

Hanson Sun
jsun33@students.cs.ubc.ca

Eitan Klinger
ekling01@students.cs.ubc.ca

Arthur Guo
aguo07@students.cs.ubc.ca

Abstract

We investigate whether Kolmogorov-Arnold Networks (KANs) can be integrated into graph neural networks (GNNs) to produce interpretable models of n-body physical systems. We introduce Graph KANs (GKANs), a hybrid architecture that replaces MLP components in message-passing GNNs with learnable spline-based edge functions. We evaluate GKANs on simulated n-body dynamics under several force laws and assess both predictive performance and interpretability through sparsity and network visualization. Our results are largely negative: GKANs are consistently harder to optimize than MLP baselines, and standard regularization techniques fail to produce interpretable sparse networks. We attribute this primarily to the interaction between spline-based edge functions and message passing, which amplifies gradient instability. Despite this, we find that physics-informed feature engineering can recover baseline GNN performance with a 99.5% reduction in model size, and that smoother force laws are substantially easier to learn—suggesting that GKANs may still hold promise under the right inductive biases.

1 Introduction

Many scientific systems can be modeled as interacting objects governed by underlying physical laws. A key goal in machine learning for science is not only to predict these systems accurately, but also to recover interpretable structure from the learned model.

We study the n-body problem as a testbed for this goal. Given the initial states of several bodies, the task is to predict their future trajectories under pairwise forces. Prior work has shown that graph neural networks (GNNs) are a strong fit because they naturally represent objects as nodes and interactions as edges. In particular, Cranmer et al. 2020 showed that a GNN trained on simulated n-body dynamics can be combined with symbolic regression to recover analytical force laws.

A limitation of this approach is that the GNN itself is not directly interpretable. The learned representation is still a black box, and symbolic regression is a separate post-hoc step. Recent work on Kolmogorov-Arnold Networks (KANs) suggests a possible alternative. KANs replace standard MLP-style node activations with learnable univariate functions on edges, which can make the learned model easier to inspect. Liu, Wang, et al. 2024 showed that KANs can be effective while remaining more transparent than standard MLPs.

In this project, we ask whether this idea can be extended to relational physical systems. We introduce a hybrid Graph KAN (GKAN) architecture, where KAN layers replace MLP components inside a message-passing GNN. We evaluate this model on simulated n-body dynamics and compare it against standard GNN baselines, with a focus on both prediction quality and interpretability. However, whether KANs integrate well with message passing is not obvious—the two components impose different structural demands on optimization, and our results reveal this tension in practice.

Course project for UBC CPSC 440/550, April 2026.

2 Related Work

This project builds on three main lines of work: graph-based physics learning, symbolic discovery, and interpretable neural architectures.

Cranmer et al. 2020 is the closest prior work. They trained GNNs on simulated n-body systems and used symbolic regression to recover physical laws from learned interaction functions. Their results show that graph models can discover useful structure, but interpretability comes only after a separate extraction step.

Other work has strengthened graph models for physical simulation. Thangamuthu et al. 2023 compared several physics-informed GNNs across multiple dynamical systems, while Dynami-CAL GraphNet (Sharma and Fink 2025) adds conservation and symmetry constraints directly into message passing. These methods improve performance, but do not directly address whether the learned interactions are easy to interpret.

KANs provide a different route to interpretability. Liu, Wang, et al. 2024 showed that KANs can represent scientific relationships through learnable univariate functions, and KAN 2.0 adds multiplicative structure that is especially relevant for physical equations. Related continuous-time work such as KAN-ODEs (Koenig, Kim, and Deng 2024) and Graph KAN-ODEs (Cappi et al. 2025) further motivate combining KANs with dynamical systems and graph structure.

3 Methodology

We evaluate GKANs on synthetic n-body simulation data generated with a leapfrog integrator. The main dataset uses Newtonian gravity,

$$a_i = G \sum_{j \neq i} m_j \frac{s_j - s_i}{\|s_j - s_i\|^3}$$

with additional force laws for generalization tests. Each example consists of initial positions, velocities, and masses, with targets given by future trajectories. We compare a GNN baseline against a GKAN, trained on identical data, loss function, and optimizer. We evaluate both predictive performance on held-out simulations and interpretability through network sparsity and visualization of learned edge functions.

Our experiments address four questions: whether GKANs match GNN performance at comparable parameter counts; whether sparse regularization improves interpretability; whether physically motivated minimal architectures are sufficient; and whether these trends generalize across force laws.

3.1 Baseline GNN

We follow the baseline GNN architecture from Cranmer et al. 2020, a node MLP of $(110 \to 300 \to 300 \to 2)$ and a message MLP of $(10 \to 300 \to 300 \to 300 \to 100)$. Note that we compute the loss as the *mean* over individual samples rather than the *sum* (as in Cranmer et al. 2020). The model is trained without regularization using an L2 loss, optimized with Adam at a learning rate of 10^{-3}.

3.2 GKAN Architecture

Our GKAN replaces the MLP components of the baseline GCN with KAN layers. Specifically, the message MLP is replaced with a KAN message network and the node update MLP with a KAN node network. Each KAN edge learns a univariate B-spline function with a fixed grid size of `G` and spline order `k`, giving it significantly more interpretable structure than a MLP layer followed by an activation.

We expect the node KAN to learn how to aggregate messages into a final acceleration vector, while the edge KANs learn the pairwise interaction functions. We hypothesize that regularization will encourage sparsity and interpretability, and that smaller KAN architectures may still perform well if they capture the right inductive biases.

3.3 Regularization

Sparsification can be achieved in KAN layers through L1 regularization of weights, as in traditional MLP layers, though an additional entropy regularizer is often needed to make sufficiently sparse KANs (Liu, Wang, et al. 2024). This suite of experiments serves to both validate regularization patterns found in traditional KANs, as well as find ideal regularization hyperparameters to encourage sparse and interpretable networks. To do so, we run a sweep through different magnitudes of L1 regularization, entropy regularization, and both combined, comparing the models to each other, as well as a control with no regularization. Doing so helps identify the efficacy of different regularizations in creating sparser models. Additionally, a model utilizing sparse initialization is also trained, which forces the networks to be sparse before training.

Each GKAN network uses a node network of 3 layers $(7 \to 8 \to 2)$, a message network of 4 layers $(10 \to 12 \to 12 \to 2)$, and is trained over 50 epochs. Once all models are trained, they are compared in both training loss and validation loss against a test set, as well as visually compared for how sparse the networks are. The ideal regularization approach is determined by which hyperparameters provide a relatively sparse and interpretable network without sacrificing performance on the validation set.

3.4 How small KAN we get?

The Kolmogorov-Arnold (KA) representation theorem states that any continuous multivariate function can be decomposed into a finite number of univariate functions and addition operations (Liu, Wang, et al. 2024).

$$f(x_{1\ldots n}) = \sum_{i=0}^{2n} \Phi_i \left(\sum_{j=1}^{n} \phi_{i,j}(x_j) \right)$$

This suggests the existence of models with significantly fewer parameters that can still represent the function we want to learn. In this section, we design and evaluate several smaller models to determine if the GKAN can perform well without extreme overparameterization.

To design a smaller model, we work backwards from a proposed "feasible route" of function compositions and additions that yields the desired force law. In other words, we set layer widths and depths based on the number of arithmetic steps needed to decompose the desired force law into univariate functions by hand. We then observe whether the model is capable of recovering the desired force law by "undoing" our decomposition.

To illustrate this process, Figure 12 is one example of a feasible route. In this example, the message network consists of 4 layers $(10 \to 6 \to 3 \to 4 \to 2)$ because the acceleration vector can be computed in 4 arithmetic steps; the node update network consists of just 1 layer $(7 \to 2)$ because it only needs to aggregate acceleration vectors from the messages.

In this section, we also experiment with feature engineering by augmenting the dataset with computed features. Specifically, we are interested in computed features that allow us to "skip steps" during the decomposition process. For example, in the decomposition described by Figure 12, it is necessary to compute the intermediate values Δx and Δy in the first layer. By providing these as inputs, we can eliminate 1 layer.

In total, we designed and evaluated 4 smaller models with several combinations of computed features as listed in Table 1. For comparison, the baseline GNN model based on Cranmer et al. 2020 has 300 hidden units and 427 002 parameters.

Table 1: The smaller models.

experiment name	message network	parameters	computed features
`small`	4 layers $(10 \to 6 \to 3 \to 4 \to 2)$	2220	
`small_rel_pos`	3 layers $(12 \to 5 \to 4 \to 2)$	2020	Δx, Δy
`small_rel_dsq`	3 layers $(13 \to 5 \to 4 \to 2)$	2112	Δx, Δy, $\lVert r \rVert^{+2}$
`small_rel_idc`	3 layers $(13 \to 4 \to 4 \to 2)$	1824	Δx, Δy, $\lVert r \rVert^{-3}$

The node update network always consists of 1 layer $(7 \to 2)$. The models are all trained for 50 epochs on the same dataset, which consists of 200 training trajectories and 20 validation trajectories. For

Table 2: Unregularized loss values for regularization experiments

L1-norm	Entropy	Training Loss	Validation Loss
0	0	1.36×10^{-1}	1.30×10^{-1}
1.0×10^{-4}	0	1.34×10^{-1}	1.30×10^{1}
1.0×10^{-3}	0	1.30×10^{-1}	1.29×10^{-1}
0	1.0×10^{-4}	1.20×10^{-1}	1.13×10^{-1}
0	1.0×10^{-1}	1.17×10^{-1}	1.14×10^{-1}
1.0×10^{-4}	1.0×10^{-4}	1.24×10^{-1}	1.17×10^{-1}
1.0×10^{-3}	1.0×10^{-3}	1.22×10^{-1}	1.17×10^{-1}
1.0×10^{-1}	1.0×10^{-1}	1.20×10^{-1}	1.18×10^{-1}
1.0×10^{1}	1.0×10^{1}	1.25×10^{-1}	1.19×10^{-1}
1.0×10^{2}	1.0×10^{2}	1.16×10^{-1}	1.12×10^{-1}

exact details on the data generation process and training regime, as well as an outline of the feasible route for each model, refer to the appendices.

3.5 Different Force Functions

Utilizing the derived small model above, we also test the GKAN's ability to learn different force functions. In particular, we replace the gravitational force with the following three force functions:

$$a_{\text{lin}} = G \sum_{j \neq i} m_j \frac{s_j - s_i}{\|s_j - s_i\|^2}, \quad a_{\text{hooke}} = G \sum_{j \neq i} \frac{s_j - s_i}{m_i}, \quad a_{\text{nice}} = G \sum_{j \neq i} \exp(\|s_j - s_i\|^2) \frac{s_j - s_i}{m_i}$$

a_{lin} is a linear gravity function that has a $1/r$ fall-off instead of $1/r^2$. a_{hooke} is a Hooke's law function that has a linear relationship with distance. a_{nice} is an arbitrary function with a nice gradient. We also ran similar regularization experiments on the different force functions to see if the same patterns hold.

4 Results and Discussion

4.1 Baseline GNN

The baseline GNN model achieves a training loss of 1.783×10^{-2} and a validation loss of 1.289×10^{-2} after 50 epochs. This serves as a reference point for evaluating the performance of the GKAN models.

4.2 Regularization

Unregularized training loss and validation loss for each combination of L1-norm and entropy regularizations in Table 2 display marginal differences between values. The largest gap in validation losses between two of the models is 0.08×10^{-1}, comparing the control (0 L1-norm, 0 entropy) to the maximal regularization (1×10^2 L1-norm, 1×10^2 entropy). The sparse initialization is the only model to not follow this trend, with training loss 3.95×10^{-1} and validation loss 3.57×10^{-1}.

None of the networks produced by each model are interpretable through visual inspection, even with 1.0×10^{-2} regularization, which did not have a significantly sparser network compared to the control. Node and message networks are available in the appendix (Figure 6).

From Liu, Wang, et al. 2024, we expected that indivdually L1-norm and entropy would be insufficient. However, contrary to our expectations, even the combined regularization did not produce significantly sparser networks or better validation losses. The differences between losses in the experiments is easily explained as noise, and none of the regularization techniques encouraged sparse networks.

This may be explained by sparse learning difficulty. Learning the sparse representation of the gravitational n-body is difficult. Using L1 loss on the difference in output accelerations may not be sufficient the model to learn "physically" important structure (eg. energy conservation, symmetries, etc.) that would encourage sparsity. We may find better performance by incorporating physical structure directly into the loss function. Physics-informed GNNs such as Dynami-CAL GraphNet

(Sharma and Fink 2025) enforce conservation of linear and angular momentum through equivariant message passing, effectively constraining the hypothesis space to physically consistent solutions. In our setting, such constraints could encourage sparser, more interpretable representations while also improving optimization stability and long-term rollout accuracy.

4.3 How small KAN we get?

Recall that we designed and evaluated 4 smaller models. The training loss and validation loss are summarized in Table 3.

Table 3: The smaller models.

experiment name	training loss	validation loss
`baseline`	1.592×10^{-2}	1.281×10^{-2}
`small`	1.130×10^{-1}	1.185×10^{-1}
`small_rel_pos`	1.873×10^{-2}	1.639×10^{-2}
`small_rel_dsq`	1.960×10^{-2}	1.172×10^{-2}
`small_rel_idc`	2.594×10^{-2}	2.073×10^{-2}

The `small` GKAN model without any computed features performs about 1 order of magnitude worse compared to the baseline GNN. This alone is quite impressive when considering the difference in model size.

Furthermore, by introducing simple computed features such as Δx and Δy (which would not be unreasonable to include, even without *a priori* knowledge of the desired force law), we observe significantly improved performance on par with the baseline—with the additional benefit of a 99.5% reduction in model size. This suggests that the GKAN could be a highly efficient architecture for domain-informed use cases.

Looking at the visualized networks in Figures 9–10, it seems that the `small` model with no feature engineering did not learn as effectively. The message and node update networks are dense, which suggests that the model did not learn to ignore features such as velocity which are not relevant to the desired force law. The introduction of computed features (obtensibly) acts as a sufficiently strong inductive bias—the models with feature engineering have sparser networks which emphasize the relevant mass- and displacement-related features. In conjunction with the improvement in the training and validation loss, this supports the idea that computed features can increase the expressive power of a model despite adding no new information (Liu, Ma, et al. 2024).

The utility of the GKAN in terms of interpretability is still unclear. A cross-section visualization (perturbing one input at a time) of the `small_rel_pos` model as shown in Figure 11 indicates that the model learned approximately correct mathematical relationships overall. However, this type of analysis is equally possible with traditional GNN architectures.

The unique property of the GKAN is that it is possible to perform symbolic regression per-layer, then compose the symbolic expressions analytically. Unfortunately, in our case, while the overall result was reasonable, the smaller models did not learn the exact per-layer transformations we had hoped for (as outlined in our feasible routes). This is not surprising—there may be an infinite number of valid decompositions for any given function—but it complicates interpretability, as the size of the symbolic expression increases exponentially with network depth. Ultimately, we were not able to produce interpretable symbolic expressions for the learned force laws.

4.4 Different force functions

The relative training and validation loss decreases the GKAN models on 10 epochs of training are in Table 4. In terms of reguarlization results, we see the same patterns with little difference in sparsity. With the only exception being Hooke's law, which has a notably more sparse network (Figure 15).

We observe a clear trend: GKANs train more effectively on smoother, better-conditioned force laws. Both training and validation loss reductions increase as the target interaction becomes less singular.

Gravity performs worst, with training loss decreasing by less than 7% over 10 epochs. This is unsurprising: the $1/r^2$ dependence introduces near-singular behavior at small separations, which is difficult

Table 4: Losses of different force functions. Gravity is trained with no min-separation constraint for fair comparison. See Figure 14 for the loss plots.

experiment name	init train loss	final train loss	init val loss	final val loss
`Gravity`	2.571×10^0	2.396×10^0	2.143×10^0	1.952×10^0
`Linear`	1.883×10^0	1.301×10^0	1.250×10^0	0.764×10^0
`Hooke`	0.752×10^0	0.300×10^0	0.225×10^0	0.141×10^0
`Nice`	7.015×10^{-2}	3.878×10^{-2}	3.721×10^{-2}	2.11×10^{-2}

for B-spline edges to represent efficiently, as their piecewise polynomial basis approximates smooth functions well but struggles with sharp or unbounded derivatives (de Boor 1972). Replacing this with the $1/r$ linear gravity law already yields a substantial improvement, with training loss dropping by $\sim 31\%$, suggesting that reducing the degree of singularity meaningfully aids optimization.

Hooke's law improves further still, achieving a $\sim 60\%$ training loss reduction. Because the interaction is linear in displacement with bounded derivatives, the model requires less functional composition and fewer active spline regions to represent it accurately. This is also reflected in the regularization results, where Hooke's law (Figure b) is the only force function that produces a noticeably sparser network. The "nice" force function, despite being nonlinear, achieves comparable relative loss reductions ($\sim 45\%$ on validation) and maintains the lowest absolute losses—likely because its exponential profile avoids the distance-magnitude and reciprocal-scaling terms that make gravity challenging.

In combination, these results suggest that GKAN optimization is sensitive to the analytic structure of the target force law. Interpretability and sparsity also benefit from smoother targets: simpler force laws not only converge faster, but compress more naturally into compact GKAN representations.

4.5 KAN't Optimize

Beyond interpretability, a persistent challenge in our experiments was optimization. GKANs were consistently harder to train than MLP-based GNN baselines: loss decreased slowly, converged to worse minima, and was sensitive to initialization. This was surprising given that Liu, Wang, et al. 2024 and Liu, Ma, et al. 2024 demonstrate effective KAN training on gravitational systems—however, their setting trains a single KAN directly on force inputs and outputs, without the added complexity of message passing.

We suspect message passing is the key differentiator. Errors in learned edge functions propagate across the graph, amplifying the gradient instability that spline-based functions already exhibit. This is consistent with known pathologies in deep GNNs such as oversmoothing and oversquashing (Arroyo et al. 2025). We tested Adam, L-BFGS, warmup schedules, and alternating optimization, but none consistently improved training, suggesting that this difficulty may be intrinsic to the GKAN architecture in message-passing settings.

5 Conclusion

We introduced GKANs, an architecture combining KAN layers with GNNs, and evaluated them on n-body dynamics. Our results were largely negative: GKANs were consistently harder to optimize than MLP-based GNN baselines, regularization failed to produce interpretable sparse networks, and training loss had mediocre decreases. This was surprising given that prior work demonstrates KANs can recover physical laws effectively—though notably in simpler, non-relational settings without message passing. Our experiments do leave us hopeful: GKANs trained with feature engineering showed good performance with a 99.5% reduction in model size, and smoother force laws were substantially easier to learn, hinting that the architecture has real potential under the right conditions.

Many directions remain unexplored. Pruning low-weight edges during training, physically-informed loss functions enforcing conservation laws, and decoupled optimization schedules that prioritize accuracy before sparsity are all natural next steps. Resolving the gradient degradation introduced by message passing—perhaps through equivariant or symmetry-constrained architectures—may be the most critical open problem for making GKANs practical.

So KAN it learn? *Kind of.*

References

Álvaro Arroyo, Alessio Gravina, Benjamin Gutteridge, Federico Barbero, Claudio Gallicchio, Xiaowen Dong, Michael Bronstein, and Pierre Vandergheynst (2025). *On Vanishing Gradients, Over-Smoothing, and Over-Squashing in GNNs: Bridging Recurrent and Graph Learning*. arXiv: 2502.10818 [cs.LG]. URL: https://arxiv.org/abs/2502.10818.

Riccardo Cappi, Paolo Frazzetto, Nicolò Navarin, and Alessandro Sperduti (2025). "Unveiling the Actual Performance of Neural-based Models for Equation Discovery on Graph Dynamical Systems." arXiv: 2508.18173 [cs.LG]. URL: https://arxiv.org/abs/2508.18173.

Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho (2020). *Discovering Symbolic Models from Deep Learning with Inductive Biases*. arXiv: 2006.11287 [cs.LG]. URL: https://arxiv.org/abs/2006.11287.

Carl de Boor (1972). "On calculating with B-splines." *Journal of Approximation Theory* 6.1, pp. 50–62. ISSN: 0021-9045. DOI: https://doi.org/10.1016/0021-9045(72)90080-9. URL: https://www.sciencedirect.com/science/article/pii/0021904572900809.

Benjamin C. Koenig, Suyong Kim, and Sili Deng (Dec. 2024). "KAN-ODEs: Kolmogorov–Arnold network ordinary differential equations for learning dynamical systems and hidden physics." *Computer Methods in Applied Mechanics and Engineering* 432, p. 117397. ISSN: 0045-7825. DOI: 10.1016/j.cma.2024.117397. URL: http://dx.doi.org/10.1016/j.cma.2024.117397.

Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark (2024). *KAN 2.0: Kolmogorov-Arnold Networks Meet Science*. arXiv: 2408.10205 [cs.LG]. URL: https://arxiv.org/abs/2408.10205.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark (2024). *KAN: Kolmogorov-Arnold Networks*. arXiv: 2404.19756 [cs.LG]. URL: https://arxiv.org/abs/2404.19756.

Vinay Sharma and Olga Fink (2025). "Dynami-CAL GraphNet: A Physics-Informed Graph Neural Network Conserving Linear and Angular Momentum for Dynamical Systems." arXiv: 2501.07373 [cs.LG]. URL: https://arxiv.org/abs/2501.07373.

Abishek Thangamuthu, Gunjan Kumar, Suresh Bishnoi, Ravinder Bhattoo, N M Anoop Krishnan, and Sayan Ranu (2023). *Unravelling the Performance of Physics-informed Graph Neural Networks for Dynamical Systems*. arXiv: 2211.05520 [cs.LG]. URL: https://arxiv.org/abs/2211.05520.

A Supplementary material

A.1 Code and Experiments

All of our code and experiments are available at `https://github.com/Hanson-Sun/3body-gkan` (not Jupyter notebooks thankfully). The repository includes the training and evaluation code for all models, as well as all the information to reproduce our experiments. Specific experiment configurations can be found in the `experiments.yaml` file.

There are several additional features that we implemented but did not have time to test fully and include in our report. These include:

- Importance score based pruning of final models to help visualization
- MultKan support, which is a variation of KAN that includes multiplication nodes in addition to addition and function composition.
- Reguarlization schedules that increase regularization magnitudes over the course of training to encourage learning first, then sparsity.
- Incomplete support for Hamiltonian GNNs.
- Large scale GKAN experiments with 500k parameters.

The repository also includes animated visualizations showcasing the rollouts (forward simulation) for selected models, which are linked in Table 5.

Table 5: Hyperlinks to rollouts for selected models.

experiment name	hyperlink
baseline	[...]/outputs/baseline/rollout_comparison.gif
small	[...]/outputs/small/rollout_comparison.gif
small_rel_pos	[...]/outputs/small_rel_pos/rollout_comparison.gif
small_rel_dsq	[...]/outputs/small_rel_pos_dsq/rollout_comparison.gif
small_rel_idc	[...]/outputs/small_rel_pos_idc/rollout_comparison.gif

A.2 Additional Visualizations and Results

This section will hold any additional figures, plots, and visualizations related to the experiments that we could not fit into the report.

A.2.1 Regularization

figures/reg/no_regularization_loss.png

Figure 1: Training and validation loss over 50 epochs for the regularization control.

figures/reg/l1_1e-4_loss.png

(a) L1-norm 1×10^{-4}

figures/reg/l1_1e-3_loss.png

(b) L1-norm $\times 10^{-3}$

Figure 2: Training and validation loss over 50 epochs for L1-norm regularization experiments.

figures/reg/entropy_1e-4_loss.png

(a) Entropy 1×10^{-4}

figures/reg/entropy_1e-3_loss.png

(b) Entropy 1×10^{-3}

Figure 3: Training and validation loss over 50 epochs for Entropy regularization experiments.

figures/reg/combined_1e-4_loss.png

(a) L1-norm and Entropy 1×10^{-4}

figures/reg/combined_1e-3_loss.png

(b) L1-norm and Entropy 1×10^{-3}

figures/reg/combined_1e-1_loss.png

(c) L1-norm and Entropy 1×10^{-1}

figures/reg/combined_1e1_loss.png

(d) L1-norm and Entropy 1×10^{1}

figures/reg/combined_1e2_loss.png

(e) L1-norm and Entropy 1×10^{2}

Figure 4: Training and validation loss over 50 epochs for combined L1-norm and Entropy regularization experiments.

figures/reg/sparse_init_loss.png

Figure 5: Training and validation loss over 50 epochs with sparse initialization.

figures/reg/com1e2_node_network.png

(a) Node network 1×10^2 combined regularization

figures/reg/com1e2_node_network.png

(b) Message network 1×10^2 combined regularization

figures/reg/control_node_network.png

(c) Node network with no regularization

figures/reg/control_msg_network.png

(d) Message network with no regularization

Figure 6: Node and message network examples for high regularization and no regularization.

A.2.2 How Small KAN We Get?

figures/baseline/gnn_loss.png

Figure 7: The baseline.

figures/small/kan_loss.png

(a) small

figures/small_rel_pos/kan_loss.png

(b) small_rel_pos

figures/small_rel_pos_dsq/kan_loss.png

(c) small_rel_dsq

figures/small_rel_pos_idc/kan_loss.png

(d) small_rel_idc

Figure 8: Training and validation loss for the smaller models.

figures/small/kan_msg_network.png

(a) small

figures/small_rel_pos/kan_msg_network.png

(b) small_rel_pos

figures/small_rel_pos_dsq/kan_msg_network.png

(c) small_rel_dsq

figures/small_rel_pos_idc/kan_msg_network.png

(d) small_rel_idc

Figure 9: Message network visualizations for the smaller models.

figures/small/kan_node_network.png

(a) small

figures/small_rel_pos/kan_node_network.png

(b) small_rel_pos

figures/small_rel_pos_dsq/kan_node_network.png

(c) small_rel_dsq

figures/small_rel_pos_idc/kan_node_network.png

(d) small_rel_idc

Figure 10: Node update network visualizations for the smaller models.

figures/small_rel_pos/msg_kan_responses.png

(a) Message network

figures/small_rel_pos/node_kan_responses.png

(b) Node update network

Figure 11: The cross-section for small_rel_pos.

input	m_i, $s_{x,i}$, $s_{y,i}$, $v_{x,i}$, $v_{y,i}$, m_j, $s_{x,j}$, $s_{y,j}$, $v_{x,j}$, $v_{y,j}$
1	$h_{1,1} = s_{x,j} - s_{x,i} \propto \Delta x$ $h_{1,2} = s_{y,j} - s_{y,i} \propto \Delta y$ $h_{1,3} = m_j + s_{x,j} - s_{x,i} \propto m_j + \Delta x$ $h_{1,4} = m_j - s_{x,j} + s_{x,i} \propto m_j - \Delta x$ $h_{1,5} = m_j + s_{y,j} - s_{y,i} \propto m_j + \Delta y$ $h_{1,6} = m_j - s_{y,j} + s_{y,i} \propto m_j - \Delta y$
2	$h_{2,1} = h_{1,1}^2 + h_{1,2}^2 \propto \lVert r \rVert^2$ $h_{2,2} = h_{1,3}^2 - h_{1,4}^2 \propto m_j \Delta x$ $h_{2,3} = h_{1,5}^2 - h_{1,6}^2 \propto m_j \Delta y$
3	$h_{3,1} = h_{2,1}^{-1.5} + h_{2,2} \propto \lVert r \rVert^{-3} + m_j \Delta x$ $h_{3,2} = h_{2,1}^{-1.5} - h_{2,2} \propto \lVert r \rVert^{-3} - m_j \Delta x$ $h_{3,3} = h_{2,1}^{-1.5} + h_{2,3} \propto \lVert r \rVert^{-3} + m_j \Delta y$ $h_{3,4} = h_{2,1}^{-1.5} - h_{2,3} \propto \lVert r \rVert^{-3} - m_j \Delta y$
4	$h_{4,1} = h_{3,1}^2 - h_{3,2}^2 \propto \lVert r \rVert^{-3} m_j \Delta x$ $h_{4,2} = h_{3,3}^2 - h_{3,4}^2 \propto \lVert r \rVert^{-3} m_j \Delta y$
output	$a_{x,i} = h_{4,1}$, $a_{y,i} = h_{4,2}$

Figure 12: The feasible route for small.

input	m_i, $s_{x,i}$, $s_{y,i}$, $v_{x,i}$, $v_{y,i}$, m_j, $s_{x,j}$, $s_{y,j}$, $v_{x,j}$, $v_{y,j}$, Δx, Δy
1	$h_{1,1} = \Delta x^2 + \Delta y^2 \propto \lVert r \rVert^2$ $h_{1,2} = m_j + \Delta x$ $h_{1,3} = m_j - \Delta x$ $h_{1,4} = m_j + \Delta y$ $h_{1,5} = m_j - \Delta y$
2	$h_{2,1} = h_{1,1}^{-1.5} + h_{1,2}^2 - h_{1,3}^2 \propto \lVert r \rVert^{-3} + m_j \Delta x$ $h_{2,2} = h_{1,1}^{-1.5} - h_{1,2}^2 + h_{1,3}^2 \propto \lVert r \rVert^{-3} - m_j \Delta x$ $h_{2,3} = h_{1,1}^{-1.5} + h_{1,4}^2 - h_{1,5}^2 \propto \lVert r \rVert^{-3} + m_j \Delta y$ $h_{2,4} = h_{1,1}^{-1.5} - h_{1,4}^2 + h_{1,5}^2 \propto \lVert r \rVert^{-3} - m_j \Delta y$
3	$h_{3,1} = h_{2,1}^2 - h_{2,2}^2 \propto \lVert r \rVert^{-3} m_j \Delta x$ $h_{3,2} = h_{2,3}^2 - h_{2,4}^2 \propto \lVert r \rVert^{-3} m_j \Delta y$
output	$a_{x,i} = h_{3,1}$, $a_{y,i} = h_{3,2}$

(a) `small_rel_pos`

input	m_i, $s_{x,i}$, $s_{y,i}$, $v_{x,i}$, $v_{y,i}$, m_j, $s_{x,j}$, $s_{y,j}$, $v_{x,j}$, $v_{y,j}$, Δx, Δy, $\lVert r \rVert^2$
1	$h_{1,1} = \lVert r \rVert^2$ $h_{1,2} = m_j + \Delta x$ $h_{1,3} = m_j - \Delta x$ $h_{1,4} = m_j + \Delta y$ $h_{1,5} = m_j - \Delta y$
2	$h_{2,1} = h_{1,1}^{-1.5} + h_{1,2}^2 - h_{1,3}^2 \propto \lVert r \rVert^{-3} + m_j \Delta x$ $h_{2,2} = h_{1,1}^{-1.5} - h_{1,2}^2 + h_{1,3}^2 \propto \lVert r \rVert^{-3} - m_j \Delta x$ $h_{2,3} = h_{1,1}^{-1.5} + h_{1,4}^2 - h_{1,5}^2 \propto \lVert r \rVert^{-3} + m_j \Delta y$ $h_{2,4} = h_{1,1}^{-1.5} - h_{1,4}^2 + h_{1,5}^2 \propto \lVert r \rVert^{-3} - m_j \Delta y$
3	$h_{3,1} = h_{2,1}^2 - h_{2,2}^2 \propto \lVert r \rVert^{-3} m_j \Delta x$ $h_{3,2} = h_{2,3}^2 - h_{2,4}^2 \propto \lVert r \rVert^{-3} m_j \Delta y$
output	$a_{x,i} = h_{3,1}$, $a_{y,i} = h_{3,2}$

(b) `small_rel_dsq`

input	m_i, $s_{x,i}$, $s_{y,i}$, $v_{x,i}$, $v_{y,i}$, m_j, $s_{x,j}$, $s_{y,j}$, $v_{x,j}$, $v_{y,j}$, Δx, Δy, $\lVert r \rVert^{-3}$
1	$h_{1,1} = m_j + \lVert r \rVert^{-3}$ $h_{1,2} = m_j - \lVert r \rVert^{-3}$ $h_{1,3} = \Delta x$ $h_{1,4} = \Delta y$
2	$h_{2,1} = h_{1,1}^2 - h_{1,2}^2 + h_{1,3} \propto m_j \lVert r \rVert^{-3} + \Delta x$ $h_{2,2} = h_{1,1}^2 - h_{1,2}^2 - h_{1,3} \propto m_j \lVert r \rVert^{-3} - \Delta x$ $h_{2,3} = h_{1,1}^2 - h_{1,2}^2 + h_{1,4} \propto m_j \lVert r \rVert^{-3} + \Delta y$ $h_{2,4} = h_{1,1}^2 - h_{1,2}^2 - h_{1,4} \propto m_j \lVert r \rVert^{-3} - \Delta y$
3	$h_{3,1} = h_{2,1}^2 - h_{2,2}^2 \propto m_j \lVert r \rVert^{-3} \Delta x$ $h_{3,2} = h_{2,3}^2 - h_{2,4}^2 \propto m_j \lVert r \rVert^{-3} \Delta y$
output	$a_{x,i} = h_{3,1}$, $a_{y,i} = h_{3,2}$

(c) `small_rel_idc`

Figure 13: Feasible routes for the smaller models with feature engineering.

A.2.3 Different Force Functions

figures/different_function_losses/min_gravity_figure.pngfigures/different_function_losses/min_linear_gravit

(a) Normal gravity, no min-separation

(b) Linear gravity

figures/different_function_losses/min_hooke_loss.pngfigures/different_function_losses/min_nice_loss.png

(c) Hooke's law

(d) Nice force

Figure 14: Training and validation loss over 10 epochs for different force functions. All trained without regularization.

figures/different_function_kans/min_gravity_sparse/kan_msg_network.pngfigures/different_function_kans/min_hooke_sparse/ka

(a) Normal gravity

(b) Hooke's law

figures/different_function_kans/min_nice_sparse/kan_msg_network.png

(c) Nice force

Figure 15: Message network visualizations for different force functions. Trained with regularization for 50 epochs.

figures/different_function_kans/min_gravity_sparse/kan_node_network.png

(a) Normal gravity

figures/different_function_kans/min_hooke_sparse/kan_node_network.png

(b) Hooke's law

figures/different_function_kans/min_nice_sparse/kan_node_network.png

(c) Nice force

Figure 16: Node network visualizations for different force functions. Trained with regularization for 50 epochs.